

Final Paradigmas Lab POO

1 de Julio 2026

Instrucciones Generales

- Para codear el examen solo podrán utilizar algún editor de texto plano o IDE (sin autocompletado con IA). No utilizar teléfonos durante el examen.
- Pueden buscar información en Google pero no pueden utilizar asistentes de IA de ninguna clase. Es obligatorio deshabilitar los resúmenes de IA que ofrece google introduciendo el comando “-IA” en todas las consultas a Google. Alternativamente pueden utilizar un buscador clásico como ecosia.
- Pueden utilizar sus propias laptops o alguna máquina del laboratorio que tenga instalado sbt.
- Pueden consultar y reutilizar el código que escribieron en el laboratorio 2, pero **solo deben entregar los archivos indicados al final**.
- Solo pueden usar las **librerías** ya incluidas en el build.sbt del esqueleto. No pueden agregar nuevas dependencias.

El esqueleto contiene dos proyectos: el proyecto principal del examen y un servidor mock de Reddit (`reddit-mock/`). **Deben tener ambos corriendo al mismo tiempo**, cada uno en una terminal separada.

Paso 1 — Levantar el servidor mock de Reddit

Para evitar que Reddit bloquee las peticiones durante el examen, el esqueleto apunta a un servidor local que simula la API de Reddit.

En una terminal, desde la carpeta `reddit-mock/`:

```
None  
sbt run
```

Deberían ver:

```
None
Fake Reddit API running on http://localhost:8123
Press Ctrl+C (or stop the process) to shut down.
```

El servidor queda corriendo en el puerto 8123 y sirve los posts de los subreddits `r/scala`, `r/programming` y `r/learnpython` desde archivos JSON locales. **No cerrarlo durante el examen.**

Paso 2 — Ejecutar el proyecto

En otra terminal, desde la carpeta raíz del esqueleto:

```
None
sbt run
```

Verán algo como el siguiente output:

```
None
Diccionario cargado: 65 entidades.

Descargando posts de: http://localhost:8123/r/scala/.json

=====
http://localhost:8123/r/scala/.json
=====
Post: "This week in #Scala (Apr 27, 2026)"
Entidades detectadas:
  [ProgrammingLanguage] Scala
Post: "try it out: Scone media player build with Scala.js, Calico, and Electron"
Entidades detectadas:
  [ProgrammingLanguage] Scala
Post: "sbt-steps 0.3.0 released: streamline your CI using the power of sbt!"
Entidades detectadas:
  (sin entidades detectadas)
Post: "Spark observability tool that works for production debugging?"
Entidades detectadas:
  (sin entidades detectadas)
Post: "Rust for Scala and Haskell Developers: A Surprisingly Familiar Journey by
Pawel Szulc"
Entidades detectadas:
  [ProgrammingLanguage] Scala
  [ProgrammingLanguage] Haskell
  [ProgrammingLanguage] Rust
  ...
```

Ejercicio 1: implementación de alias

En los posts, una misma entidad puede aparecer con nombres alternativos: `JS` en lugar de `JavaScript`, `GH` en lugar de `GitHub`, `Guido` en lugar de `Guido van Rossum`. El ejercicio consiste en agregar soporte de alias a la jerarquía.

a) Agregar en `NamedEntity` el método concreto:

```
None
def aliases: List[String] = List.empty
```

b) Sobrecribir `aliases` en `Technology` para que retorne la lista de alias cargada desde un archivo `data/aliases.txt` con el formato:

```
None
JavaScript=JS,ECMAScript
Python=Py
C++=CPP
```

Cada línea tiene la forma `entidad=alias1,alias2,...`. Si la entidad no tiene entradas en el archivo, retorna `List.empty`. La carga del archivo puede hacerse con `FileIO.readLine` ya disponible. Obviamente, crear el archivo correspondiente `data/aliases.txt`.

c) Agregar en `NamedEntity` el método concreto que combine `text` y `aliases`:

```
None
def allNames: List[String] = text :: aliases
```

d) Modificar `Analyzer.detectEntities` para que use `allNames` en lugar de solo `text` al buscar coincidencias: si cualquier nombre o alias de la entidad aparece en el texto, la entidad es detectada.

e) Modificar `Formatters.formatNERResult` para que cuando una entidad fue detectada por un alias (no por su `text` canónico), lo indique:

```
None
[Technology] JavaScript (detectado como: JS)
```

Ejercicio 2: implementación de relevancia numérica

Cada tipo de entidad nombrada define un grado de relevancia que luego se usará para filtrar y ordenar.

a) Agregar en `NamedEntity` el método concreto:

```
None  
def relevanceScore: Int = 5
```

Luego, para cada tipo de entidad nombrada implementar su relevancia con los siguientes valores:

Tipo	relevanceScore
Person	8
Organization	3
University	6
Place	2
Technology	4
ProgrammingLanguage	9

b) Agregar en `Analyzer`:

```
None  
def detectAboveThreshold(text: String, dictionary:  
List[NamedEntity], threshold: Int): List[NamedEntity]
```

Que retorne solo las entidades cuyo `relevanceScore >= threshold` y que hagan match en el texto. El threshold se pasa como parámetro (no hardcodeado).

c) Agregar en `Formatters`:

None

```
def formatScoredResult(postTitle: String, entities:
List[NamedEntity]): String
```

Que muestre las entidades ordenadas de mayor a menor según su `relevanceScore`, incluyendo el score en cada línea:

None

```
Post: "Rust for Scala and Haskell Developers"
Entidades detectadas (por relevancia):
  [9] ProgrammingLanguage: Scala
  [9] ProgrammingLanguage: Haskell
  [9] ProgrammingLanguage: Rust
```

Modificar `Main` para usar `detectAboveThreshold` con threshold 5 y `formatScoredResult` al mostrar cada post. Las estadísticas globales al final siguen usando `detectEntities` sin filtro.

Entrega

Para entregar el examen deben subir TODOS los archivos modificados por ustedes a través del formulario de entrega adjuntado un solo archivo comprimido que los incluya a todos:

- `*.scala`
- `data/aliases.txt`

Para generar el archivo comprimido pueden usar un comando similar al siguiente:

None

```
tar -czvf parcial2.tar.xz data/aliases.txt src/main/scala/*.scala
```

Deberán recibir un mail de confirmación, no se retiren del aula hasta recibirlo, ya que, este representa su único comprobante de entrega de examen.